

ZScript Interpreter

OOP Design Document

Name: Muhammad Abdullah | Roll Number: BSCS25017

1. Value Hierarchy

Class	Field / Key methods
Value	+to_string() +is_truthy() +type_name()
NumberValue	raw: double — zero is falsy
StringValue	raw: string — empty string is falsy
BooleanValue	raw: bool — false is falsy

Each subclass overrides the three virtual methods to define its own conversion, truthiness, and type label.

2. Expression Hierarchy

Base class: **Expression** — subclasses: NumberLiteral, StringLiteral, BooleanLiteral, VariableExpr, BinaryExpr.

The single polymorphic method is **evaluate(Environment&): Value***. Literals return a fresh value object; VariableExpr performs a lookup; BinaryExpr evaluates both operands then applies the operator.

3. Statement Hierarchy

All statements override **execute(Environment&): void**.

Statement	Key Fields	Action
PrintStmt	expr	Evaluates & prints
LetStmt	name, init	Defines new variable
AssignStmt	name, value	Updates existing variable
IfStmt	condition, then_branch, else_branch	Conditional branching
WhileStmt	condition, body	Looping
BlockStmt	first	Creates child scope

4. Environment Class

Category	Detail
Fields	names[50]: string values[50]: Value* count: int parent: Environment*
Methods	define(name, value) — introduce new binding assign(name, value) — walk up chain to update lookup(name) — walk up chain to read create_child() — new scope with this as parent

Scoping is handled by a parent-pointer chain. BlockStatement always creates a child environment so inner variables do not leak outward.

5. Lexer and Parser

Lexer	Parser
source: string tokens[100]: Token current, line: int token_count: int	statements[256]: Statement* stmt_count: int tokens[100]: Token current: int
tokenize() scan_next() advance(), peek() is_at_end()	parse(): Statement** parse_statement() parse_let(), parse_if() parse_while(), parse_block() parse_expression()

The Token struct carries a **TokenType**, a lexeme string, and a line number. The Lexer converts raw source text into a flat token array. The Parser consumes that array and builds the AST as a **fixed-size array of Statement* pointers** (statements[256]) rather than a linked list — stmt_count tracks how many entries are populated. Execution iterates over indices 0 ... stmt_count-1 instead of following next pointers.

6. Error Class

ZScriptError carries a **message: string** and a **line: int**. It is thrown at three points:

- Lexer — unrecognised character in source
- Parser — unexpected token or malformed grammar
- Runtime — type error or undefined variable

7. Code Flow Walkthrough

Sample program

```
let x = 10;
let y = 20;
```

```

if ( x < y ) { print ("y is bigger") ; }
while ( x > 0) { x = x - 1; }
print ( x ) ;

```

Step 1 — Initialisation

main() constructs a Lexer with the source string and calls tokenize().

Step 2 — Lexer produces tokens

scan_next_token() reads characters one at a time and emits Token structs:

```

tokens[0] LET "let"
tokens[1] IDENTIFIER "x"
tokens[2] EQUAL "="
tokens[3] NUMBER "10"
tokens[4] SEMICOLON ";"
...

```

Step 3 — Parser builds the AST (array-based)

parse() loops over tokens and calls specialised helpers, appending each Statement* into the **statements[]** array and incrementing stmt_count. No linked-list next pointers are used. The resulting array:

```

statements[0] = LetStmt("x", NumberLiteral(10))
statements[1] = LetStmt("y", NumberLiteral(20))
statements[2] = IfStmt(BinaryExpr(x < y), BlockStmt{PrintStmt("y is bigger")})
statements[3] = WhileStmt(BinaryExpr(x > 0), BlockStmt{AssignStmt("x", x-1)})
statements[4] = PrintStmt(Variable("x"))
stmt_count = 5

```

Step 4 — Interpreter loop

```

Environment global_env;
for (int i = 0; i < stmt_count; ++i) {
statements[i]->execute(global_env); // virtual dispatch
}

```

Virtual dispatch does all the work. The loop iterates by index — no pointer chasing required.

Step 5 — LetStatement executes (x = 10)

```

// LetStatement::execute()
Value* val = initializer->evaluate(env); // NumberLiteral -> NumberValue(10)
env.define(name, val);

```

After both let statements the global environment holds:

```

names[0] = "x" values[0] = NumberValue(10)
names[1] = "y" values[1] = NumberValue(20) count = 2

```

Step 6 — IfStatement executes

```
// IfStatement::execute()
Value* result = condition->evaluate(env); // BinaryExpr: 10 < 20 -> BooleanValue(true)
if (result->is_truthy())
then_branch->execute(env); // BlockStmt creates child scope
```

BlockStatement creates a child environment and runs PrintStatement, which outputs *y is bigger*.

Step 7 — WhileStatement executes (10 iterations)

```
// WhileStatement::execute()
while (true) {
Value* result = condition->evaluate(env);
if (!result->is_truthy()) break;
body->execute(env); // AssignStmt: x = x - 1
}
```

env.assign() walks up from the child scope to global_env to update x. When x reaches 0, condition 0 > 0 returns BooleanValue(false) and the loop exits.

Step 8 — Final PrintStatement

```
// PrintStatement::execute()
Value* val = expression->evaluate(env); // VariableExpr("x") -> NumberValue(0)
cout << val->to_string() << endl; // Output: 0
```

Step 9 — Program ends

The for-loop index reaches stmt_count (5); the interpreter exits and main() returns 0.

Terminal output: y is bigger 0